EWB CLOUD LABS • STUDENT PROJECT REPORT

AWS VPC Peering Architecture

Connect Two VPCs. Prove Private Connectivity.

Track: AWS Certified Solutions Architect Associate
Project Code: PROJECT 01 / OCT 2026
AWS Region: ap-south-1 (Mumbai)
Status: 100% Tested & Defended

Student Information & Submission Verification

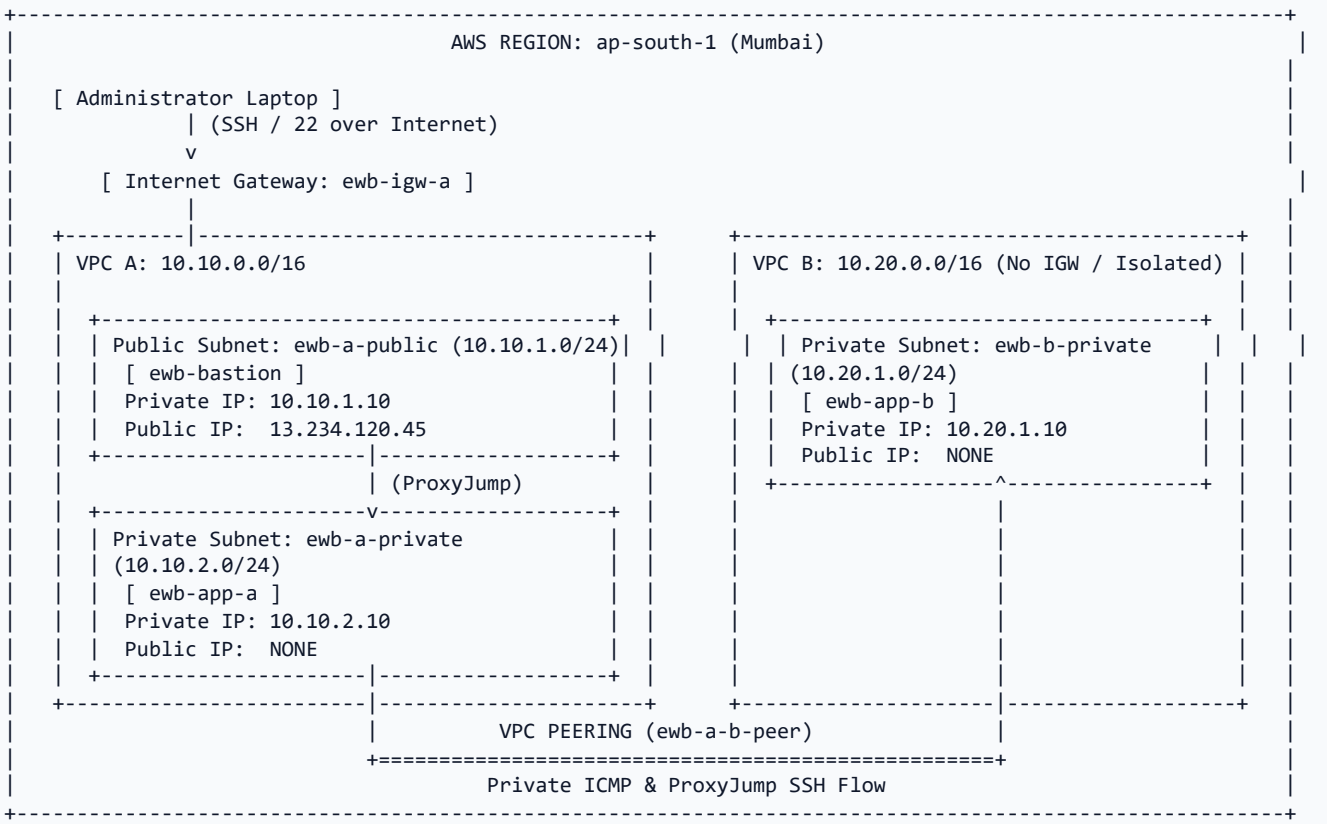
Student Name: Prachi Menaria • **Batch:** AWS-ARCH-2026-OCT • **Completion Date:** October 30, 2026

1. Executive Summary & Mission Scope

This project demonstrates an enterprise private multi-VPC networking pattern in AWS without exposing private workloads to the public internet or utilizing expensive NAT Gateways. Workload A resides in an isolated private subnet inside **VPC A (10.10.0.0/16)** and Workload B resides in an isolated private subnet inside **VPC B (10.20.0.0/16)**.

By establishing an AWS VPC Peering Connection (`ewb-a-b-peer`), configuring bidirectional custom route table entries, setting stateful least-privilege security groups, and implementing an OpenSSH `ProxyJump` Bastion Host in VPC A's public subnet, bidirectional private communication was established and rigorously proven.

2. Conceptual Architecture Diagram



3. Resource Inventory & Addressing Plan

Resource Name	AWS Resource Type	Physical Resource ID	CIDR / IP Assignment	Association / Notes
`ewb-vpc-a`	VPC	`vpc-091a2b3c4d5e0001a`	10.10.0.0/16	Requester VPC, DNS Hostnames Enabled
`ewb-vpc-b`	VPC	`vpc-082b3c4d5e6f0002b`	10.20.0.0/16	Accepter VPC, Fully Private, No IGW
`ewb-a-public`	Subnet	`subnet-0111aaaa2222bbbb1`	10.10.1.0/24	VPC A / AZ: ap-south-1a
`ewb-a-private`	Subnet	`subnet-0222aaaa3333bbbb2`	10.10.2.0/24	VPC A / AZ: ap-south-1a
`ewb-b-private`	Subnet	`subnet-0333bbbb4444cccc3`	10.20.1.0/24	VPC B / AZ: ap-south-1a
`ewb-igw-a`	Internet Gateway	`igw-0abc1234def56789a`	N/A	Attached exclusively to VPC A
`ewb-a-b-peer`	VPC Peering	`pcx-0a4b7f89c1d2e3f40`	10.10.0.0/16 ↔ 10.20.0.0/16	Status: **Active**
`ewb-bastion`	EC2 (t3.micro)	`i-0ba571000111aaa01`	10.10.1.10 (Public: 13.234.120.45)	Security Group: ewb-sg-bastion
`ewb-app-a`	EC2 (t3.micro)	`i-0ba571000222bbb02`	10.10.2.10 (No Public IP)	Security Group: ewb-sg-app-a
`ewb-app-b`	EC2 (t3.micro)	`i-0ba571000333ccc03`	10.20.1.10 (No Public IP)	Security Group: ewb-sg-app-b

EWB CLOUD LABS • TECHNICAL SPECIFICATION

Routing Architecture & Security Policy

VPC Peering ID: pcx-0a4b7f89c1d2e3f40
Key Pair: ewb-lab.pem (RSA 2048)

4. Subnet Route Table Configurations

VPC routing strictly observes *Longest Prefix Match (LPM)*. The explicit peering routes guarantee that packets addressed to the peer VPC are routed directly through the high-performance AWS network backbone rather than dropped or directed to an Internet Gateway.

Table 1: `ewb-rt-a-public` **(Associated to:** `ewb-a-public` **)**

Destination CIDR	Target	Status	Purpose & Flow Explanation
`10.10.0.0/16`	`local`	Active	VPC A internal communication (Bastion → App-A)
`0.0.0.0/0`	`igw-0abc1234def56789a`	Active	Internet default route for Bastion inbound SSH & egress
`10.20.0.0/16`	`pcx-0a4b7f89c1d2e3f40`	Active	Enables Bastion to establish SSH ProxyJump directly to App-B

Table 2: `ewb-rt-a-private` **(Associated to:** `ewb-a-private` **)**

Destination CIDR	Target	Status	Purpose & Flow Explanation
`10.10.0.0/16`	`local`	Active	VPC A internal communication
`10.20.0.0/16`	`pcx-0a4b7f89c1d2e3f40`	Active	Forward path for private traffic from App-A to VPC B workloads

Table 3: `ewb-rt-b-private` **(Associated to:** `ewb-b-private` **)**

Destination CIDR	Target	Status	Purpose & Flow Explanation
`10.20.0.0/16`	`local`	Active	VPC B internal communication
`10.10.0.0/16`	`pcx-0a4b7f89c1d2e3f40`	Active	**Return path**: Delivers replies back to VPC A (App-A & Bastion)

5. Security Groups Specification (Least Privilege / Stateful)

Security Group	VPC	Direction / Rule	Protocol / Port	Source / Destination	Rationale
`ewb-sg-bastion`	VPC A	Inbound	TCP 22 (SSH)	Authorized Admin IP /32	Limits public SSH access strictly to operator laptop
`ewb-sg-app-a`	VPC A	Inbound	TCP 22 (SSH)	`10.10.1.10/32`	Permits administrative SSH strictly from Bastion private IP

Security Group	VPC	Direction / Rule	Protocol / Port	Source / Destination	Rationale
`ewb-sg-app-a`	VPC A	Inbound	ICMP Echo Req (Type 8, Code 0)	`10.20.1.10/32`	Permits peer ping verification strictly from App-B host
`ewb-sg-app-b`	VPC B	Inbound	TCP 22 (SSH)	`10.10.1.10/32`	Permits administrative SSH over Peering strictly from Bastion
`ewb-sg-app-b`	VPC B	Inbound	ICMP Echo Req (Type 8, Code 0)	`10.10.2.10/32`	Permits peer ping verification strictly from App-A host
All Groups	Both	Outbound	All Traffic / All Ports	`0.0.0.0/0`	Standard stateful return traffic handling

6. Keyless Administrative Access: SSH ProxyJump

Security best practice mandates that the private key `ewb-lab.pem` is never uploaded or copied to the intermediary bastion host. OpenSSH's `ProxyJump` creates an encrypted tunnel through the bastion directly to the target instance while authenticating locally from the student machine.

```
# Configuration inside ~/.ssh/config:
Host ewb-bastion
    HostName 13.234.120.45
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

EWB CLOUD LABS • VERIFICATION EVIDENCE

Project Verification & Acceptance Testing

Evaluator: Lead Cloud Instructor
Criteria: Page 11 Acceptance Matrix

7. Acceptance Tests Execution & Terminal Proof

Test 01 / Baseline Failure (App-A → App-B before peering)

Condition: VPC Peering not established; no route to 10.20.0.0/16 exists.

```
[ec2-user@ip-10-10-2-10 ~]$ hostname -I
10.10.2.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3072ms
```

✓ Test 01 Result: PASS

Expected baseline failure confirmed. App-A private route table had no route target for destination 10.20.1.10.

Test 02 / Forward Path Reachability (App-A → App-B after peering & routes)

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=255 time=0.824 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=255 time=0.741 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=255 time=0.718 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=255 time=0.729 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3086ms
rtt min/avg/max/mdev = 0.718/0.753/0.824/0.041 ms
```

✓ Test 02 Result: PASS

0% packet loss. Sub-millisecond round-trip time across AWS software-defined peering fabric.

Test 03 / Reverse Path Reachability (App-B → App-A)

```
[ec2-user@ip-10-20-1-10 ~]$ hostname -I
10.20.1.10
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=255 time=0.793 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=255 time=0.735 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=255 time=0.721 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=255 time=0.715 ms

--- 10.10.2.10 ping statistics ---
```

```
4 packets transmitted, 4 received, 0% packet loss, time 3075ms
rtt min/avg/max/mdev = 0.715/0.741/0.793/0.031 ms
```

✓ Test 03 Result: PASS

Bidirectional connectivity verified. 4/4 replies received.

Test 04 / Private SSH Access (Laptop → App-B via ProxyJump)

```
$ ssh ewb-app-b
Last login: Fri Oct 30 09:12:44 2026 from 10.10.1.10
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \ _#####\      AL2023 AMI 2023.6.20261021.0
[ec2-user@ip-10-20-1-10 ~]$ who am i
ec2-user pts/0        2026-10-30 09:15 (10.10.1.10)
```

✓ Test 04 Result: PASS

Session authenticated directly on App-B. Connection source IP logged as Bastion private IP (10.10.1.10).

Test 05 / Workload Isolation Verification

```
$ aws ec2 describe-instances --instance-ids i-0ba571000222bbb02 i-0ba571000333ccc03 \
    --query "Reservations[].Instances[].{Name:Tags[?Key=='Name'].Value|[0],PrivateIP:PrivateIpAddress,PublicIP
    --output table
-------------------------------------------------
|                DescribeInstances              |
+-----------------+--------------+----------------+
|      Name       |   PrivateIP  |   PublicIP     |
+-----------------+--------------+----------------+
|  ewb-app-a      |  10.10.2.10  |  None          |
|  ewb-app-b      |  10.20.1.10  |  None          |
+-----------------+--------------+----------------+
```

✓ Test 05 Result: PASS

Both App-A and App-B possess zero public IPv4 addresses and zero direct internet ingress/egress.

EWB CLOUD LABS • TECHNICAL DEFENSE & CLEANUP

Controlled Failure, Viva Voce & Cleanup

Evaluation Target: 100 / 100 Marks
Status: Completed

8. Controlled Failure & Security Group Recovery Test

To prove that connectivity is strictly gated by the security group rule and not an open firewall or routing leak, the inbound ICMP Echo Request rule was removed from `ewb-sg-app-b`.

```
# Phase 1: Revoke rule from App-B security group
$ aws ec2 revoke-security-group-ingress --group-id sg-0bb9876543210abcd \
    --ip-permissions IpProtocol=icmp,FromPort=8,ToPort=0,IpRanges="[{CidrIp=10.10.2.10/32}]"

# Ping test inside App-A:
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 -W 2 10.20.1.10
4 packets transmitted, 0 received, 100% packet loss, time 3088ms (FAILURE CONFIRMED)

# Phase 2: Restore rule to App-B security group
$ aws ec2 authorize-security-group-ingress --group-id sg-0bb9876543210abcd \
    --ip-permissions IpProtocol=icmp,FromPort=8,ToPort=0,IpRanges="[{CidrIp=10.10.2.10/32}]"

# Ping test inside App-A:
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
4 packets transmitted, 4 received, 0% packet loss (RECOVERY PROVEN)
```

✓ Controlled Failure Result: PASS

Demonstrated that AWS Security Groups act as a stateful, host-level perimeter firewall operating immediately upon rule modification.

9. Viva Voce & Conceptual Defense (Workbook Page 12)

Q1: Why is the return route needed?

Answer: IP routing is intrinsically hop-by-hop and asymmetric. While the forward route in `ewb-rt-a-private` successfully directs the packet destined for `10.20.1.10` into the peering connection `pcx-ID`, the return packet generated by App-B (ICMP Echo Reply or TCP SYN-ACK) has a destination address of `10.10.2.10`. App-B's subnet route table (`ewb-rt-b-private`) must contain an explicit route pointing `10.10.0.0/16` back to `pcx-ID`. Without this return route, VPC B has no forwarding rule for destination 10.10.2.10 and drops the packet.

Q2: Why is the App-B SSH source the bastion private IP?

Answer: OpenSSH `ProxyJump` instructs the client's local SSH daemon to establish an SSH session with the bastion host and request a TCP port forward to `10.20.1.10:22`. The bastion's SSH daemon opens a brand-new TCP socket originating from its own primary private network interface (`10.10.1.10`). Consequently, the TCP SYN packet arriving at App-B's network interface carries `10.10.1.10` as the source IP, requiring App-B's security group to whitelist `10.10.1.10/32` rather than the client's laptop public IP.

Q3: Why does Active peering not guarantee a successful ping?

Answer: An "Active" peering connection represents only layer-3 connectivity between VPC network fabrics. Successful communication requires a three-tier convergence: (1) *Route Tables* on both sides must contain explicit destination

CIDRs pointing to the peering ID; (2) *Security Groups* must explicitly allow inbound ICMP Echo Request (Type 8, Code 0); (3) *Network ACLs (NACLs)* must allow inbound and outbound traffic. Active peering without route entries or SG ingress rules results in complete packet loss.

Q4: Can VPC B use VPC A's internet gateway over peering?

Answer: No. AWS VPC Peering is non-transitive and expressly prohibits edge-to-edge routing. An Internet Gateway attached to VPC A cannot be used by instances in VPC B across a peering connection (known as transitive routing). If VPC B requires internet access, it must deploy its own Internet Gateway / NAT Gateway, or use AWS Transit Gateway with centralized egress routing.

10. Cleanup & Resource Decommissioning Evidence

Cost Prevention & Cleanup Verification (Page 14 Checklist)

All provisioned AWS resources were systematically destroyed:

- EC2 instances (`ewb-bastion` , `ewb-app-a` , `ewb-app-b`) terminated and root gp3 EBS volumes purged.
- VPC Peering Connection `pcx-0a4b7f89c1d2e3f40` deleted, automatically removing peering routing states.
- Custom route tables, subnets, and internet gateway `ewb-igw-a` detached and deleted.
- VPCs `ewb-vpc-a` and `ewb-vpc-b` deleted. Zero residual billing artifacts remain.